

The Software Decides; the Chip Only Says Who

Offline DSM: A High-Level Explainer for Bitcoin Readers

SOFTWARE AUTHORITY, HARDWARE IDENTITY — whole-state consumption, a three-factor witness,
and no chip as judge

Brandon “Cryptskii” Ramsay
Irrefutable Labs Inc.

Abstract

Offline DSM is the mode in which two devices settle a transfer with no connectivity, no storage set, and no third party in reach. Every offline-cash design before it has answered the double-spend question the same way: put the rule inside a tamper-resistant chip, have the chip co-sign every payment, and trust it never to sign the same value twice. When the chip is broken, the money is broken.

DSM does not do this. Transfer uniqueness offline is the same software theorem that runs online. A device’s state is one resource; a transfer consumes it whole; one parent state admits exactly one accepted successor at an honest receiver. That proof mentions no chip, reads no counter, and holds with the hardware deleted.

What software cannot do is tell the enrolled device from a perfect copy of its bytes. That is the one job the hardware is given, and the only one. An offline release must be witnessed by three factors over the same message: the DSM signature from the BIP39 seed on the phone, a non-exportable Ed25519 signature from a key resident inside a TROPIC01 secure element, and a signature from a key sealed inside the RP2350 host partition that unseals only under the enrolled firmware measurement. The chip and host witnesses are minted at commit, after a one-way counter step that consumes the origin, so no signed release for an origin exists while that origin is still spendable. A single enrolled appliance therefore cannot produce two releases from one origin, even to two strangers who never meet.

The device carries two permanent identity domains. The online domain needs a phone and a seed and nothing else. The offline domain is added later, if ever, by a birth ceremony, and the two coexist. A monotone counter committed as a leaf of the device’s Sparse Merkle Tree makes offline position an explicit part of the consumed state; the physical counter in the chip merely tracks it, as a non-rewind floor, a stale-image tripwire, and an exposure cap.

This explainer accompanies the *Software Authority, Hardware Identity* specification [1] and the DSM papers it builds on [2, 3, 4]. It is written for a Bitcoin reader and organised around that reader’s objections rather than the order in which the mechanism is constructed. Shaded boxes state how Bitcoin, hardware wallets, Lightning, eCash, the Chaum–Pedersen observer, and offline CBDC designs solve the corresponding problem and where this differs. Costs and boundaries are stated where they arise.

Contents

I	What Is Claimed, and Who Decides	4
1	The Central Question	4
2	The Entire Idea in One Diagram	5
3	Six Answers Before the Details	6
4	What an Offline Transfer Is	7
5	The Observer Model, Inverted	8
6	Uniqueness Is Software; Identity Is Hardware	8
7	Two Domains, One Device	9
8	Nothing Live Is Consulted	10
II	The Construction	10
9	Birth: Where the Offline Identity Comes From	11
10	The Counter Inside the Root	11
11	The Three-Factor Witness	12
12	Prepare, Commit, Emit	13
13	What the Receiver Checks	15
14	The Physical Counter: Floor, Tripwire, Budget	15
15	Power Loss	16
III	Attacks	17
16	Two Offline Recipients	17
17	The Software Clone	19
18	Rogue Firmware and the Measurement Gate	19
19	The Restored Backup	20
20	The Perfect Clone and What Happens Later	20
21	Compromise Matrix	21
IV	Assessment	21
22	Relation to Everything Else That Works	22

23 What Is Not Introduced, and What Is Not Claimed	22
24 What the Silicon Has Shown	23
25 Security Properties	24
26 The Complete Causal Architecture	25
27 Final Invariant	25
28 Conclusion	26

Part I

What Is Claimed, and Who Decides

The Bitcoin reader’s objections, in the order they occur. Everything here is stated normatively in the specification; the mechanism is built in Part II and attacked in Part III.

1 The Central Question

Every offline payment design has to answer one question:

When the payer and the payee are alone in a room with no network, what stops the payer from walking into the next room and paying the same value to someone else?

Most answers name a chip. A tamper-resistant element in the payer’s wallet holds the balance, co-signs every payment, and refuses to sign the same value twice. The Chaum–Pedersen observer of 1992 is this design; Brands’ cash sits inside it; fielded smartcard purses used it; most offline CBDC pilots still do. In every case the chip is the double-spend authority, and the security of the money is the security of the chip. DSM answers differently:

Nothing stops the payer from *constructing* a second payment. What is stopped is the second payment *becoming state*. A device’s state is one resource. A transfer consumes it whole. One parent admits one accepted successor at an honest receiver, and the receiver decides that from the bytes in front of it, without a chip’s opinion. The chip is asked one question only: is this the enrolled device?

The rule everything follows from

transfer uniqueness is a software property; hardware proves device identity and nothing else.

The hardware never authorises a transaction, never orders transactions, and never decides uniqueness. The receiver’s acceptance predicate reads no live chip state, no live host state, no raw counter, no relay session, and no boot record. It reads canonical bytes, Merkle proofs, and three signatures over one message, and it computes a verdict.

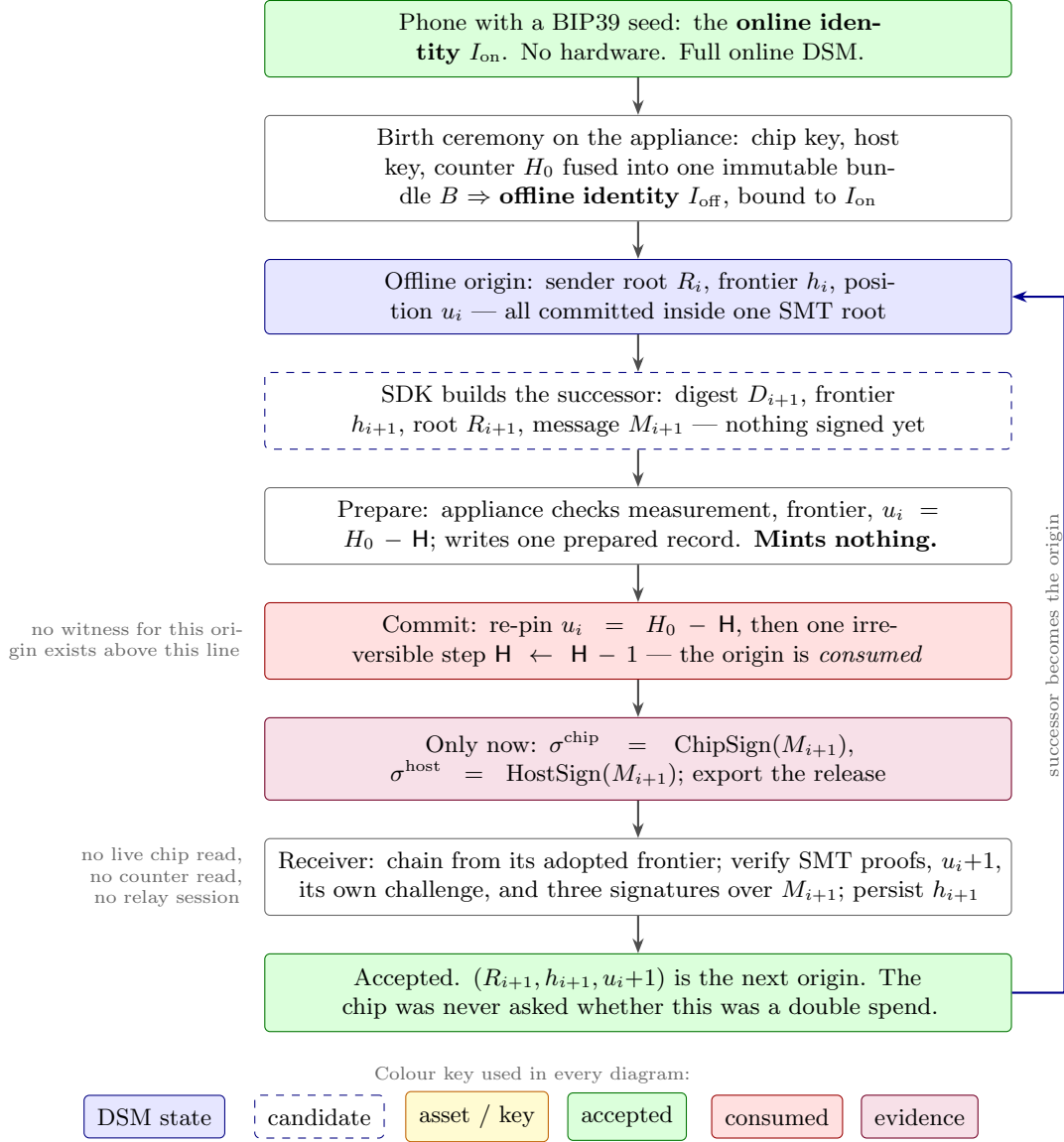
Two jobs, never conflated

Uniqueness is the question *can this successor be the one that follows this parent?* It is a property of the state predicate. It is a theorem of the DSM kernel and it is the same theorem online and offline.

Identity is the question *did this successor come from the enrolled physical device, or from a copy of its host-readable bytes?* Software cannot answer it; a perfect copy of the bytes runs the same software. This is the hardware’s job.

The previous offline specification mixed the two: it made a hardware counter the uniqueness authority and then built a session of witnessed counter reads around it to make the hardware appear to bind the transition it bracketed. Adversarial review showed a scalar counter read never binds anything, and further review showed the binding was never needed. Section 23 lists what that review removed.

2 The Entire Idea in One Diagram



Three features of that picture are the whole design.

The top band costs nothing. Online DSM is a phone and a seed. The appliance is added later, if ever, and adds a domain without disturbing anything already established.

The witnesses are born below the consumption. Prepare produces nothing that can be harvested; Commit consumes the origin with one counter step *and only then* signs. There is no interval in which a signed release for a still-spendable origin exists anywhere.

The receiver's column touches no live hardware. Every check is a computation over bytes it holds: proofs against a root, an arithmetic step, a challenge it chose, and three signatures against keys pinned at birth.

3 Six Answers Before the Details

The question the reader has	The answer, and where it is earned
What stops an offline double spend?	The same thing that stops it online: a device state is one resource, consumed whole, and one parent admits one accepted successor at an honest receiver. This is a theorem about roots, leaves, and hash chains. It reads no hardware. (Sections 6, 16)
Then what is the chip for?	Identity. A software clone holds every host-readable byte and runs the same code. Only a non-exportable key can distinguish the enrolled device from that clone. The chip and host each contribute one such key; the seed contributes the third. (Sections 5, 11)
What about two strangers who never meet?	Neither has a frontier to reject against, so this case is carried by the appliance rather than the receiver. Witnesses are minted only after the one-way counter step that consumes the origin; enrolled firmware holds one active state and one prepared record; rogue firmware cannot unseal the host signer. One appliance, one origin, one release. (Sections 12, 16)
What if the whole appliance is cloned?	A perfect live clone of all three non-exportable factors is indistinguishable by any offline-only protocol, and the paper says so. Both releases then collide at one counter position under one bundle: any party holding both decides it on sight, the split is bricked, and the count between reconciliations is capped by policy. (Sections 20, 23)
Do I need this hardware to use DSM?	No. The online domain is a seed on a phone. The offline domain is an upgrade, and the two are permanent and coexisting. (Section 7)
What does the receiver actually read?	Its own stored frontier for the holder, its own fresh challenge, SMT proofs against the sender's roots, an arithmetic check that the position advanced by exactly one, and three signatures over one message against two keys pinned in the bundle plus the holder's DSM key. Nothing live. (Section 13)

Just enough notation

Roots, frontier, position.

R_i is the sender's device SMT root before a transfer and R_{i+1} after. $h_i \rightarrow h_{i+1}$ is the *offline frontier*, a dedicated forward-only hash lineage advanced exactly once per offline transfer; it is what a counterparty tracks. u_i is the committed *anchor counter*, the software position of the offline origin, stored as a leaf of R_i . The origin of a transfer is the triple (R_i, h_i, u_i) .

Two counters.

H_0 is the value the chip's monotonic counter held at enrollment; H is its live value. The chip counts *down*: $H \leftarrow H - 1$. The software position counts *up*. They are held together by $u = H_0 - H$, checked, never assumed.

Bundle B .

The immutable enrollment digest of the offline domain. It commits the chip's per-die public identity, the chip witness key pk_{chip} , the host witness key pk_{host} , H_0 , the enrolled firmware measurement μ_{enrolled} , and a hash of a birth secret that was destroyed. $I_{\text{off}} = H(B)$ up to domain separation.

Message and witnesses.

D_{i+1} is the digest of the transition core. M_{i+1} is the *root-advance message*: a hash binding B , both roots, both frontiers, both positions, D_{i+1} , the recipient, and the receiver's challenge r_R . σ^{chip} and σ^{host} are signatures over M_{i+1} ; the DSM signature σ^{DSM} rides on the transition itself.

Release, package, Branch Proof.

The release certificate carries the fields of M_{i+1} and the two hardware signatures. The package adds the transition core, SMT proofs Π_i, Π_{i+1} , an optional Branch Proof (the chain of prior certificates from the

receiver's last-known frontier), and, at genesis only, the upgrade certificate U .

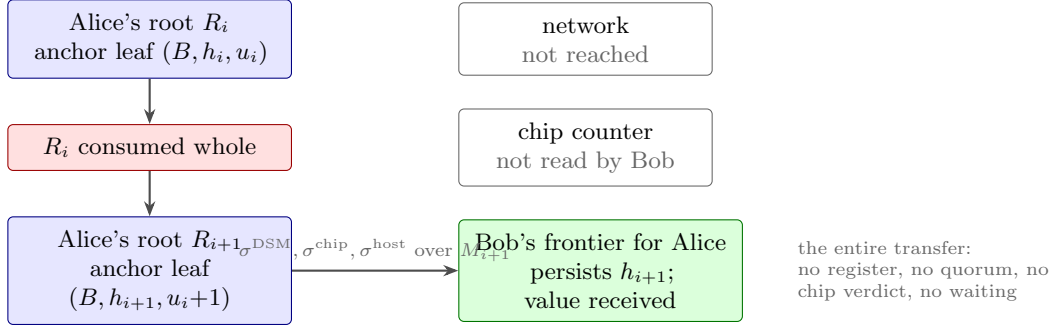
Measurement μ .

The running firmware's measurement. The host signing service unseals only when $\mu = \mu_{\text{enrolled}}$.

4 What an Offline Transfer Is

Alice holds an offline origin (R_i, h_i, u_i) and pays Bob. Both are disconnected.

That is one DSM transition and nothing else. Alice's device root R_i is consumed as a whole; the relationship leaf for $A \leftrightarrow B$, the object leaves, and the anchor leaf all move together under one root replacement to R_{i+1} . The anchor leaf goes from (B, h_i, u_i) to (B, h_{i+1}, u_i+1) . Bob verifies the transition the way any DSM receiver verifies any transition, plus two extra signatures that say the device is the device.



What this buys

Because an offline transfer is pure DSM state with two identity signatures attached, there is no counter disclosure round trip, no relay session to the chip, and no live read of anything. A first transfer to a stranger is one round trip like any other transfer, plus the upgrade certificate at genesis. Bob's verdict is computed, not received.

Key Idea

An offline DSM transfer is the online transfer with the register unreachable. Uniqueness comes from the same place. What the offline mode adds is a committed position inside the root, so the receiver can decide staleness from the state, and three identity signatures, so the receiver can decide the device is enrolled.

Bitcoin Comparison

An offline Bitcoin payment is an unconfirmed transaction handed over in person. It is exposed until it reaches the chain, and if a conflicting transaction is mined first the merchant holds nothing and can prove nothing.

An offline DSM payment is a consumed origin and an accepted successor. It is not waiting for anything, and if a conflicting successor is ever presented it collides at one position under one device identity and is decidable on sight.

Architectural consequence:

The information a Bitcoin merchant waits for is whether the payer's coin has moved. DSM commits that information as a counter inside the payer's root and hands it to the receiver as part of the payment.

Tradeoff / boundary:

Offline acceptance still depends on the receiver having, or being given, the payer's lineage: an adopted frontier or a verifying Branch Proof back to one. A stranger at first contact has neither, and that case is carried by the appliance rather than the receiver (Section 12).

5 The Observer Model, Inverted

Anyone who has looked at offline digital cash will recognise the shape of a secure element that co-signs payments. Chaum and Pedersen proposed it in 1992 as the *wallet with observer*: a tamper-resistant chip inside the user’s wallet that co-signs every payment and refuses to sign the same value twice. Brands’ cash sits in the same model. Fielded smartcard purses used it, and most recent offline CBDC and secure-element payment designs still do.

In that model the hardware is the double-spend authority. It is trusted to enforce uniqueness. That means the protocol inherits the hardware’s entire attack surface as its own: extract the key, break the counter, or emulate the chip, and double spending follows.

DSM refuses that assignment.

	Observer model	DSM offline bearer
Who enforces uniqueness?	the chip	the state predicate
What does the chip prove?	“this value was not spent”	“this is the enrolled device”
What does the receiver read?	the chip’s co-signature on the coin	three signatures over one root-advance message, and its own frontier
Break the chip and...	double spend	forge a device identity; a second successor of one origin still cannot become state

The last row is the inversion. In the observer model a hardware compromise is a protocol compromise. In DSM a hardware compromise lets an attacker impersonate a device; it does not let two successors of one origin both become realized state, because that exclusion never lived in the hardware.

Bitcoin Comparison

Bitcoin readers know hardware wallets, and know that a hardware wallet protects a key, not the ledger. If the wallet is compromised the coins can be stolen, but the UTXO set does not become inconsistent. DSM holds the same line one layer deeper. The offline anchor protects the identity of the physical device, not the uniqueness of the transfer. Uniqueness comes from the same resource-consumption theorem that runs online.

Architectural consequence:

The security argument for offline transfer does not reduce to “trust the chip.” The chip answers one narrow question, and the answer to that question cannot create a second valid consumption of a consumed origin.

Tradeoff / boundary:

A perfect live clone of every non-exportable factor is indistinguishable from the original by any offline-only protocol. That is the stated boundary of the identity claim. It is a boundary on who the device is, not on whether a transfer is unique.

6 Uniqueness Is Software; Identity Is Hardware

This section states the property the previous specification tried to buy from hardware, and separates it into two claims.

Claim one: whole-state consumption

A DSM device state is a single resource. An accepted transfer consumes the parent device root R_i entirely and yields exactly one successor R_{i+1} . There is no partial advance. The relationship leaf, the object leaves, and, for offline transfers, the anchor state leaf move together under one root replacement.

Now suppose Bob has accepted Alice’s transfer and persisted h_{i+1} as his frontier for her. Alice later presents a second transfer that claims the previous frontier h_i . Bob’s acceptance predicate requires the proposal to chain from his adopted frontier, either directly or by a Branch Proof. It does not chain directly, because $h_i \neq h_{i+1}$. And a Branch Proof from h_{i+1} back to h_i would have to be a verifying hop chain

$$h_i \rightarrow h_{i+1} \rightarrow \dots \rightarrow h_i,$$

a cycle in a forward-only hash lineage, which is a collision in H . So Bob rejects, before executing anything, without comparing anything to anything, and without reading a chip.

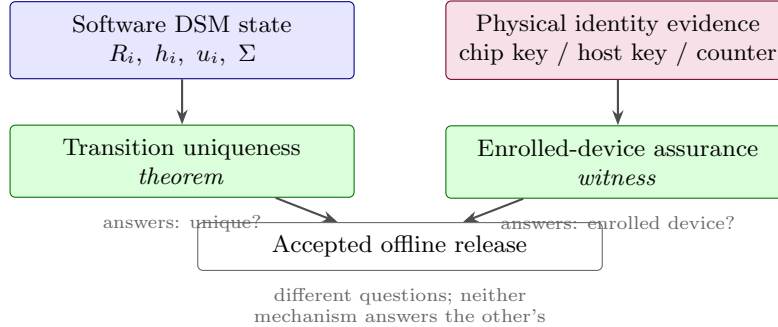
Adopted-Frontier Exclusion. For every adversary, no honest receiver accepts and executes two successors of one adopted origin. The rejection is up front, requires no reconciliation round, and reads no hardware state.

The proof examines adopted frontiers, hash lineage, SMT roots, and DSM validity only. Removing every chip read, host read, counter read, and relay session from the design does not touch it. This is the same theorem the DSM explainer proves for online operation [4]; the offline mode inherits it rather than restating it.

Claim two: the one thing software cannot see

A *software clone* holds all host-readable state: seed, keys, chain history, local database, cached proofs. It runs the same code. Against a receiver who holds no frontier for the holder, and who therefore has nothing to reject against, software alone cannot say which of two identical-looking devices is the enrolled one.

This is the residue, and it is all the hardware is asked to cover. Section 11 builds the witness; Section 12 shows why, under the enrolled firmware, one appliance cannot emit two releases from one origin even to two strangers.



Key Idea

The uniqueness theorem holds with the anchor hardware deleted. The hardware does not repeat that theorem. It answers the only question the theorem leaves open offline: did this advance come from the enrolled device running the code that respects the theorem?

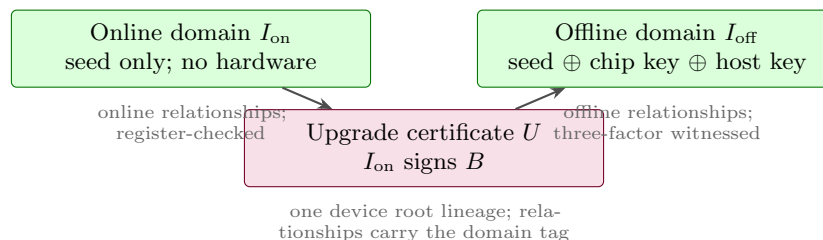
7 Two Domains, One Device

The device carries two identities in two hash-separated domains. Both are permanent. Relationships are tagged by domain when established and never migrate.

Online identity I_{on} is derived from the BIP39 seed alone. It requires no hardware, costs nothing beyond a phone, and supports all online DSM operation: relationship establishment, bilateral transfer, reconciliation.

Offline identity I_{off} exists only after appliance birth and is the hash of the anchor bundle B . Producing a valid offline release requires live cooperation of all three factors: the DSM transition is signed under seed-derived keys, and the release carries the chip and host signatures over the same root-advance message.

The **upgrade ceremony** binds the two. A device operating online-only performs appliance birth, obtains B , and publishes an upgrade certificate $U = (B, pk_{\text{on}}, \sigma^{\text{on}})$ in which the online identity signs the offline bundle. Counterparties who have themselves upgraded may then open offline relationships with the device. Nothing already established changes.



The online domain is deliberately one factor: it is the zero-hardware entry path, and phone compromise is therefore online-domain takeover. Value that matters should be held in offline-domain relationships or under policy that requires offline authority for high-value movement. A user may keep the phone, the appliance, and the seed backup physically separate; the offline domain then survives the loss or compromise of any one of them, and of any two.

Bitcoin Comparison

Bitcoin has one custody model: a key signs. A hardware wallet is the same key held more carefully. DSM has two declared identity domains with different proofs and different boundaries, and which one a relationship lives in is visible rather than implicit. The cheap one is a phone; the strong one is a phone plus an appliance that proves it is itself.

Architectural consequence:

There is no hardware cost barrier to online participation, and adding the offline domain later does not re-issue anything or migrate any relationship.

Tradeoff / boundary:

The separation constrains the user: an offline relationship needs a counterparty who has also upgraded, and the online domain's single factor is exactly as safe as the phone.

8 Nothing Live Is Consulted

A Bitcoin reader who has seen secure-element designs will expect the receiver to talk to the payer's chip: open a session, read a counter before and after, confirm the decrement happened. That machinery existed in the previous specification. It is gone, and the reason is worth stating because it is the reason the whole design is shaped the way it is.

A scalar counter read proves the chip's coordinate, freshly, over a session. It never proves the chip reached that coordinate *because of this transition*: the counter primitive does not see the message, and any binding metadata wrapped around a scalar is receiver-asserted, not chip-attested. Two receivers could witness the same physical decrement as their own. The reads were theatre, and removing them removed most of the specification.

What the receiver verifies instead is a signature over a message that names the transition, the recipient, and the receiver's own challenge, under a key that was pinned at birth and cannot leave the silicon. That signature is portable, verifiable by anyone, and needs no session.

Key Idea

The receiver does not verify "the PUF" and does not verify "the counter." It verifies a resident-key signature whose public key is pinned in B , over a message it helped construct. The PUF is the at-rest substrate for the chip identity; the signature is the offline witness.

Part II

The Construction

Birth, the counter inside the root, the three-factor witness, the prepare-commit-emit discipline, what the receiver checks, what the physical counter is for, and what happens when the power fails.

9 Birth: Where the Offline Identity Comes From

The offline domain begins with a ceremony that happens once per appliance and cannot be repeated without producing a different identity.

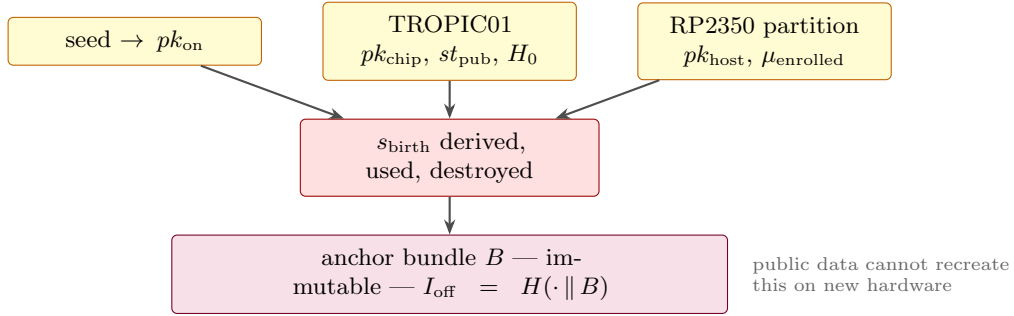
At birth the appliance gathers entropy from the RP2350 partition, a witness from the TROPIC01, host entropy, the online identity commitment, the device context, and the firmware measurement policy, and derives a *birth secret* s_{birth} . It publishes only $H(s_{\text{birth}})$, derives the initial partition seal from the secret, and then destroys the secret.

Under that seal the host generates its witness key pair; the private half never leaves the partition, and the seal unseals only when the running firmware’s measurement equals the enrolled one. Inside the TROPIC01 the chip generates its Ed25519 witness key pair in slot; the private half is not exportable through the chip’s API and is protected at rest by the die’s physically unclonable function. The chip’s monotonic counter is initialised to its maximum; that value becomes H_0 .

All of it is fused into one digest:

$$B = H(\text{DSM/anchor-bundle/v2} \parallel H(pk_{\text{on}}) \parallel st_{\text{pub}} \parallel pk_{\text{chip}} \parallel pk_{\text{host}} \parallel H_0 \parallel \text{device_id} \parallel \mu_{\text{enrolled}} \parallel \text{policy} \parallel H(s_{\text{birth}})).$$

Offline releases under a different bundle are not valid successors of state committed to B . A re-initialised counter under a new H_0 is a new bundle, a new I_{off} , and a fresh identity with no claim to the old lineage.



Key Idea

Public enrollment data is insufficient to recreate the offline identity on new hardware. A party with only public data must invert H , extract non-exportable state, or bypass the measurement gate. The enrolled counter value H_0 is *not* secret; counterparties need it to evaluate $u = H_0 - H$. What was destroyed is the birth preimage.

10 The Counter Inside the Root

The offline domain adds exactly one structural element to the base DSM device state: a monotone counter committed as a leaf of the device’s Sparse Merkle Tree. The addition is required, not decorative.

Without a committed counter, the position of a state is implicit in its root history, staleness is decidable only by walking a chain, and hardware synchronisation is not even definable. With it, position is a scalar inside the root, and the root consumes it.

The *anchor state leaf* is keyed by the bundle and commits the offline position:

$$L_i = H(\text{DSM/anchor-state/v2} \parallel B \parallel h_i \parallel u_i).$$

The *offline frontier* is a dedicated forward-only lineage advanced exactly once per offline transfer:

$$h_{i+1} = H(\text{DSM/anchor-root-advance/v2} \parallel h_i \parallel D_{i+1}).$$

What the committed counter provides

1. **Explicit position.** u_i is proven to any receiver by an SMT inclusion proof against R_i . Position is read from the state, not inferred from history.
2. **Consumption with the whole state.** The counter lives inside the root. Advancing it is not a side effect of a transfer; it is part of the single root replacement that *is* the transfer. The index cannot advance without the state, and the state cannot advance without the index.
3. **Validation before execution.** The receiver rejects a stale or incompatible proposal at acceptance time, from the state, not after the fact.
4. **Staleness on sight.** Against any known later frontier, an older state is exposed by comparing two integers.
5. **Proposal comparability.** Two proposals from one origin that claim distinct successors necessarily collide at position u_i+1 under the same leaf key. The conflict is a decidable leaf collision, not a fork requiring global ordering.
6. **A synchronisation target.** The physical counter is meaningful only because there is a committed software register for it to track. The invariant $u = H_0 - \mathbf{H}$ is a statement about this leaf.

Online transfers do not touch the counter leaf, the frontier, or the hardware counter. The SMT counter counts offline commits only.

Bitcoin Comparison

A Bitcoin transaction has no position of its own; its position is whatever block includes it, and a merchant learns the coin has moved only by watching the chain.

A DSM offline origin carries its position as a committed integer inside the root that is consumed. The successor commits u_i+1 . Two successors of one origin claim the same integer under the same identity and are distinguishable by comparing two leaves.

Architectural consequence:

The double spend is not resolved later by ordering. It is refused at acceptance by a chaining check, and if two proposals ever meet, decided by an integer comparison.

Tradeoff / boundary:

The counter is a per-device offline position, not a global sequence. It orders one device's offline commits and says nothing about anyone else's. That is the whole point, and it is also the limit.

11 The Three-Factor Witness

An offline release is witnessed by three signatures over the same message. The dependency order is a clean directed acyclic graph:

$$D_{i+1} \rightarrow h_{i+1} \rightarrow L_{i+1} \rightarrow R_{i+1} \rightarrow M_{i+1} \rightarrow \sigma^{\text{chip}}, \sigma^{\text{host}}.$$

The transition core Δ_{i+1}° holds the action, recipient identity, object, payload, old-state proofs from R_i , the counter pair (u_i, u_i+1) , and the receiver's challenge r_R . It deliberately excludes the successor root. Its digest is D_{i+1} . The root-advance message binds both sides:

$$M_{i+1} = H(\text{DSM/root-advance/v2} \parallel B \parallel R_i \parallel R_{i+1} \parallel h_i \parallel h_{i+1} \parallel u_i \parallel u_{i+1} \parallel D_{i+1} \parallel \text{recipient} \parallel r_R).$$

Factor	Where the key lives	What it says
σ^{DSM}	phone; derived from the BIP39 seed	this transition is authorised by the holder's DSM identity
σ^{chip}	TROPIC01, resident Ed25519 key generated in slot at birth; PUF-protected; never exported	this exact root advance was signed inside this physical die
σ^{host}	RP2350 secure partition; sealed at birth; unseals only when $\mu = \mu_{\text{enrolled}}$	this exact root advance was signed by the enrolled firmware, on the enrolled host, after the counter step

A complete release therefore proves live cooperation of phone, chip, and host over this exact transition, this exact recipient, and this exact challenge. Absent any factor, the corresponding signature cannot be produced. A software clone holding every host-readable byte produces no offline release at all.

Two details a careful reader should hold onto. First, the chip's signatures are not assumed deterministic: the July 2026 bench probe observed different signature bytes for the same key and message, both valid. The protocol never hashes a chip signature into any root or message; receivers verify signatures and never compare their bytes. Second, the receiver does not verify the PUF. It verifies a signature against pk_{chip} pinned in B . The PUF is why that key cannot be lifted off the die.

Key Idea

Three keys in three places, one message. The seed proves the holder; the chip proves the die; the host proves the enrolled code ran. Any two without the third produce a package the receiver refuses, and there is no two-of-three.

12 Prepare, Commit, Emit

The appliance has three states: Ready, Prepared, Committed. Offline bearer mode is enabled only when the firmware measurement matches the enrolled policy. This gate is load-bearing for the host witness, not an optional check.

Prepare mints nothing

The receiver supplies a fresh 256-bit challenge r_R . The SDK, not the appliance, constructs the successor: it computes Δ_{i+1}° , D_{i+1} , h_{i+1} , the new anchor leaf, R_{i+1} , and the SMT proofs. The appliance is not the SMT engine. It checks that the measurement matches, that it is Ready, that the supplied parent root and frontier match its active state, that $u_i = H_0 - H$, and that the fields are internally consistent. It recomputes M_{i+1} and writes one durable prepared record.

No signature is produced. No counter has moved. No release exists. Exactly one prepared record may exist per frontier.

This is the load-bearing change from the previous design. Because the chip and host witnesses are not produced at Prepare, they are never present in flash while the origin is still spendable. A malicious host cannot prepare, abort, and re-prepare to harvest multiple signed certificates for one origin, because Prepare produces nothing to harvest.

Commit consumes, then signs

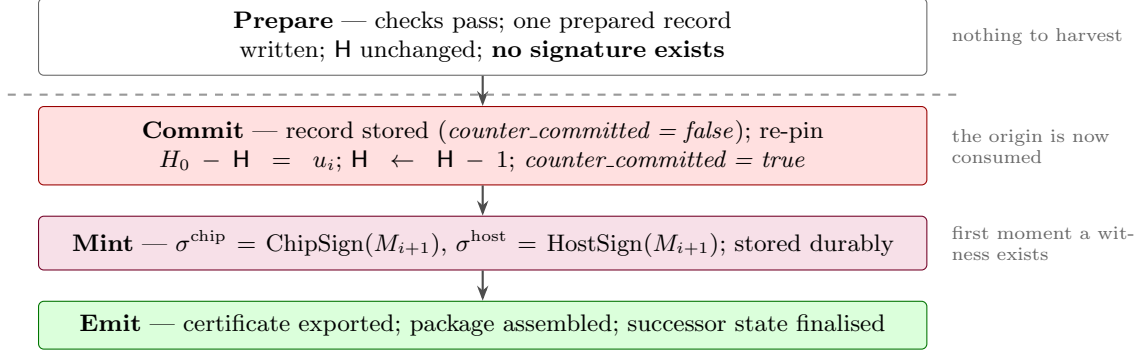
The appliance stores the committed candidate with *counter_committed* = false, re-pins $H_0 - H = u_i$ with checked arithmetic, and if that holds and $H > 0$, issues the single counter update

$$H \leftarrow H - 1,$$

marks *counter_committed* = true, and *only then* mints σ^{chip} and σ^{host} over the message fixed by the committed record. If the re-pin fails, the operation downgrades to online recovery without moving the counter and without minting. If $H = 0$ the appliance is exhausted and returns to online-only operation.

Emit only after commit

The certificate is exported only after the counter step and the minting. The SDK assembles the package. Finalisation requires $\text{Active}.u + 1 = H_0 - H$ and writes the successor active state.



Why this gives one appliance one release per origin

Suppose Alice has an enrolled appliance and wants two distinct releases from origin (R_i, h_i, u_i) , to two strangers who will never compare notes. A second release needs a second message M'_{i+1} binding the same origin but a different successor or recipient, and needs $\text{HostSign}(M'_{i+1})$ and $\text{ChipSign}(M'_{i+1})$.

No host or chip witness for the origin exists until Commit, and Commit performs the one decrement that moves the live counter from u_i to $u_i + 1$. After it, re-pinning $H_0 - H = u_i$ fails, so a second Commit at the same origin is refused and no second witness at position u_i can be minted. Preparing and aborting produces no witness. So at most one host witness ever exists for the origin, and it is the one bound into the single committed release.

If Alice runs the enrolled firmware, it exposes only the enrolled state machine: one active frontier, one prepared record per frontier, minting confined to Commit, no export before commit, recovery by re-emission. If Alice runs modified firmware, the measurement gate refuses to unseal the host signer; she can call the chip freely, but a package lacking σ^{host} fails acceptance. What remains is export of a private key, a break of the hash or signature scheme, or perfect live emulation of all three factors, and those are outside the model.

Single-Appliance First-Contact Exclusion. Under the enrolled firmware, a single enrolled appliance cannot emit two distinct valid offline releases from the same origin, even to disconnected first-contact receivers with no prior frontier for the holder.

Bitcoin Comparison

A Bitcoin wallet will happily sign two conflicting transactions from one UTXO; the chain decides which one lands. A hardware wallet's job is to keep the key from leaving, not to refuse the second signature. The DSM offline appliance refuses the second signature, not by policy inside a chip that could be lied to, but by timing: the witness is created only after the step that makes the origin unspendable, and the firmware that could create it out of order cannot unseal the key.

Architectural consequence:

Single use of the host witness is a timing property of a one-way counter, not a confidentiality property of at-rest storage. There is no window in which a signed-but-unspent release sits in flash waiting to be copied.

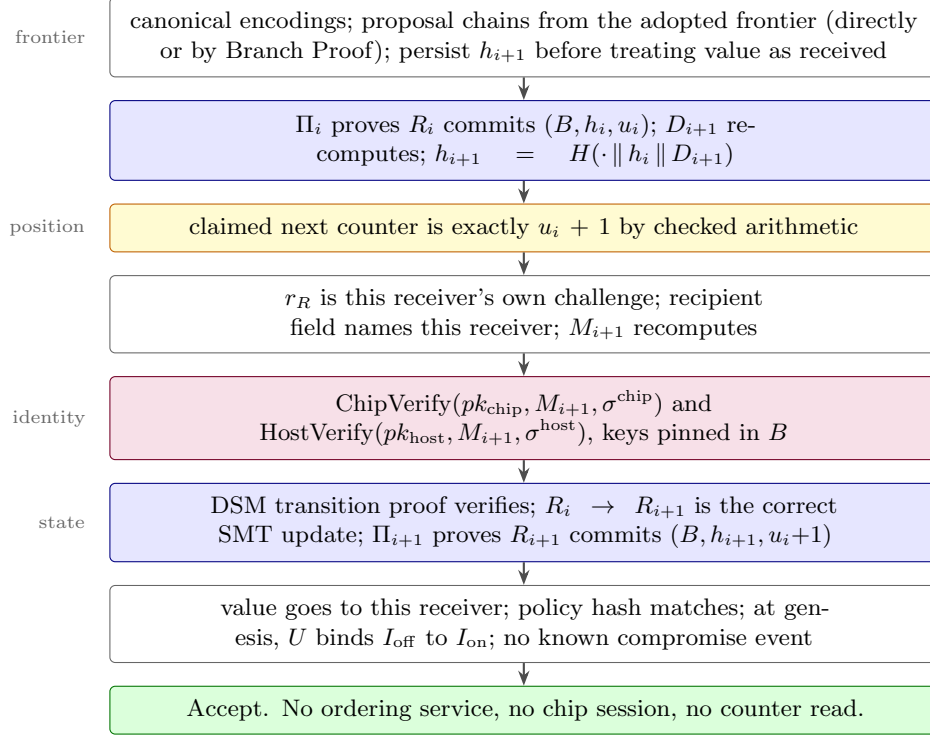
Tradeoff / boundary:

This is an emission claim about one enrolled appliance under an intact measurement boundary. It is not the uniqueness theorem, which needs none of it. Break the boundary and the emission claim is gone; the

uniqueness theorem is not (Section 20).

13 What the Receiver Checks

For each enrolled holder the receiver keeps a durable *adopted frontier*. A proposal must chain from it: either the proposal’s previous frontier equals it exactly, or the proposal carries a verifying Branch Proof from it to the proposal’s previous frontier. If no frontier exists, the relationship is at genesis and genesis adoption applies.



What does *not* appear anywhere in that list: a live chip read, a live host read, a raw counter read, a relay session, a verifier slot, a passthrough, a boot ticket, a MAC-and-destroy output, or a fused anchor head. The receiver trusts public DSM verification, its own challenge, and two pinned identity keys. It does not trust host-reported state, copied wallet files, or any unauthenticated field carried inside the release.

Authenticated counter audit exists, and it is useful, but it is outside acceptance. It cannot make an invalid release valid, cannot repair a frontier mismatch, cannot replace any of the three signatures, and cannot upgrade a package. It is diagnostic evidence, and the default receiver path never performs it.

Key Idea

The receiver’s verdict is a computation over bytes it holds, keyed by a challenge it chose. Nothing about the payer’s device is queried at acceptance; everything about it is proven.

14 The Physical Counter: Floor, Tripwire, Budget

The authoritative position register is the SMT counter: committed, proven, consumed with the state. The chip’s monotonic counter is a tracker bound to it one-to-one, one physical decrement per offline commit. It has three jobs, and none of them is acceptance authority.

1. **Non-rewind floor.** The hardware counter is monotonic and non-resettable under the enrolled H_0 . A device image restored from a backup carries an anchor leaf with $u < H_0 - H$. The appliance refuses offline operation on it, and any authenticated diagnostic read exposes it.
2. **Stale-image and split bricking.** Of two divergent lineages claiming the same identity, at most one is counter-consistent with the live chip. A diagnostic read distinguishes them; the inconsistent lineage is bricked.
3. **Offline exposure cap.** Policy bounds $u - u_{\text{rec}} \leq W$, where u_{rec} is the position at the holder's last reconciliation and W is a window committed in the policy hash. Lifetime capacity is bounded by H_0 , the cap, and flash wear; at $H = 0$ the appliance is online-only until authority rotation.

The cap is meaningful only together with the measurement gate. Enrolled firmware is what ensures one physical decrement corresponds to one committed release. If the measurement boundary is broken, the system has left the single-appliance model and entered the perfect-clone boundary.

Bitcoin Comparison

Bitcoin has no per-wallet spending budget; the chain will take whatever the key signs. Offline CBDC designs impose one inside the chip, and the chip is what enforces it.

The DSM exposure cap is a policy bound on a counter the chip tracks, checked at the appliance and at reconciliation, and it bounds *how many* offline commits can occur between reconciliations. It is a budget, not an authority.

Architectural consequence:

A holder's worst-case offline exposure between reconciliations is a number chosen in advance and committed in the bundle, not a function of how long the network stays down.

Tradeoff / boundary:

The cap counts commits, not value. Bounding value is a policy question: a production wallet may earmark an explicit offline allocation, so that device loss exposes only the loaded slice. That is a compatible extension, not part of the acceptance rule.

15 Power Loss

Power may fail between any two operations. The recovery rule is one sentence: *if a committed release exists, re-emit that same release and finalise that same successor*. Recovery never signs a new release for the same counter step.

Because witnesses are minted only at or after the counter decrement, a prepared-but-uncommitted state has no witnesses to lose and no counter step to reverse; a committed state either already holds its witnesses or has performed the decrement that fixes the single message those witnesses cover. The recovery function is total over the three appliance states and every ordering of the committed position against the live counter:

State	Observed	Outcome
Committed	counter stepped, witnesses present	re-emit the same release
Committed	counter stepped, witnesses missing	mint over the one message the record fixes; re-emit
Committed	counter not yet stepped, live position equals the record's parent	step once, mint, re-emit
Committed	anything inconsistent	downgrade to online recovery; nothing signed
Prepared	live position equals active position	cancel to Ready; nothing was minted or burned
Ready	active position <i>behind</i> the live counter	fail closed: restored stale image
Ready	active position <i>ahead</i> of the live counter	counter mismatch or wrong chip; no offline resumption
Ready	positions equal, $H > 0$	resume
any	firmware measurement fails	downgrade to online; no signing authority exposed

Bitcoin Comparison

A wallet that crashes mid-signing can usually just try again, because Bitcoin arbitrates conflicting attempts by ordering.

Here the counter step has already consumed the origin, so “try again differently” is exactly what must be impossible. Recovery is pinned to the message the committed record already fixes.

Architectural consequence:

The crash cases are decidable rather than probabilistic: re-emit the exact committed artifact, or fail closed. There is no window in which a crash lets a second, different release exist.

Tradeoff / boundary:

Fail-closed is a real outcome. A holder can end up online-only with a committed release still to be delivered, and the specification requires durable retention at each stage precisely because of that.

Part III

Attacks

Every offline-cash design in history has failed at the two-recipient attack. It gets the full treatment here, followed by the clone, the rogue firmware, the restored backup, and the case the design does not claim to defend.

16 Two Offline Recipients

Alice's appliance holds an offline origin (R_i, h_i, u_i) . She is in a place with no connectivity, and so are Bob and Carol. She pays Bob from that origin, then walks across the room and pays Carol from the same origin. Neither recipient can reach the register. Neither can reach the other. Each sees only what Alice hands them.

What each recipient sees

Bob receives an advance from (R_i, h_i, u_i) to $(R_{i+1}, h_{i+1}, u_{i+1})$, carried by three signatures over M_{i+1} , with M_{i+1} binding Bob's challenge and Bob as recipient. Bob verifies the SMT proofs, the arithmetic, his challenge, the three signatures against the keys in B , persists h_{i+1} , and accepts.

Carol is handed something that claims to be an advance from the same (R_i, h_i, u_i) to some $(R'_{i+1}, h'_{i+1}, u_i+1)$, with her challenge, naming her. She has no way to ask whether u_i has already been consumed. So the question is not what she can observe about Alice’s history. It is whether Alice can produce that object at all.

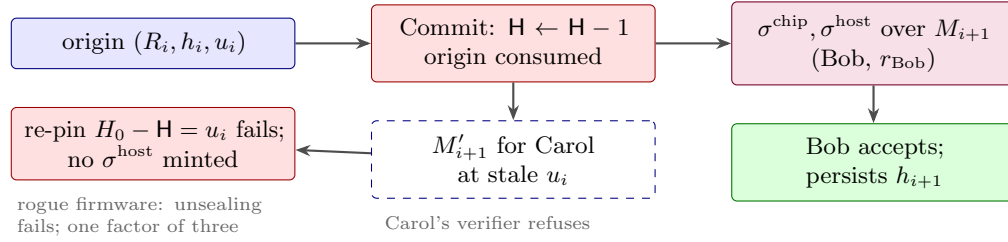
Case one: Carol already knows Alice

If Carol has ever accepted from Alice, she holds an adopted frontier h_k . Alice’s second package must chain from it. If h_k is at or later than h_{i+1} , the package claims a previous frontier Carol has already moved past, and no Branch Proof can repair it without closing a cycle in a forward-only lineage. Carol rejects at the first check. This is the software theorem, and it reads nothing.

Case two: Carol is a stranger

Carol has no frontier. She verifies the three-factor witness and the SMT update and, if they pass, adopts at genesis. So what Alice needs is a second valid witness at position u_i .

After paying Bob, the enrolled appliance’s committed position is u_i+1 and the chip’s counter has stepped once. To pay Carol from u_i , Alice needs $\text{HostSign}(M'_{i+1})$ over a message that names the stale position. The seed signature she can produce; it is software, she holds the seed, she can run modified code. The other two she cannot. No witness at u_i was minted before the decrement, and after the decrement the re-pin at u_i fails, so the enrolled firmware will not mint one. Modified firmware that would sign the stale position is not the measured firmware, so the partition refuses to unseal and there is no σ^{host} . The chip will sign for rogue firmware, but a package with one valid factor of three is refused.



This is the content of Section 12 made concrete. The software theorem says two consumptions of (R_i, h_i, u_i) cannot both be realized against a receiver who holds a frontier. The appliance discipline says one enrolled appliance cannot even *emit* the second package for a receiver who holds none.

The clone

Now suppose Alice has done the expensive thing: extracted every non-exportable factor and built a second appliance that is, at the moment of use, indistinguishable from the enrolled one. Both hold the origin at u_i . Device one pays Bob; device two pays Carol. Both witnesses verify. Both recipients accept.

This is the stated boundary, and the document does not pretend otherwise: a perfect live clone of every factor is indistinguishable from the original by any offline-only protocol. That was true of the Chaum–Pedersen observer, of every smartcard purse, and of every offline CBDC design, and it is true here. What differs is what happens next; Section 20 takes it up.

Bitcoin Comparison

An unconfirmed Bitcoin payment accepted offline is exposed until the transaction reaches the chain, and if a conflicting transaction is mined first, the merchant holds nothing and can prove nothing about who defrauded them.

An offline DSM payment is exposed only to a perfect live clone of the payer’s enrolled appliance, and if that clone is used, the defrauded recipient holds a verifiable, device-attributable proof of equivocation the moment the two packages meet.

Architectural consequence:

The offline double spend requires hardware extraction rather than a race, is unconstructible from a single

enrolled appliance, is detected at first comparison rather than never, and produces attributable evidence rather than a silent loss.

Tradeoff / boundary:

Detection is not recovery. Whether the defrauded recipient is made whole is a question for policy and the issuer, not for the state machine.

17 The Software Clone

The cheap attack first, because it is the one anyone can mount. An attacker copies the phone: seed, keys, chain history, local database, cached proofs, application state. Two hosts now hold I_{on} , and each has every byte the other has.

Offline domain. Neither copy holds the TROPIC01 resident key or the RP2350 partition key. Neither can produce σ^{chip} or σ^{host} . Neither produces an offline release at all. The copy is inert for offline purposes, however complete it is.

Online domain. Both copies can operate. Neither can double spend into the same honest receiver, because whole-state consumption admits one accepted successor per parent. The two copies may originate incompatible proposals and diverge at their first independent attempts; the identity splits. Tripwire exposes the split at first cross-contact or reconciliation, and counterparties reject both lineages pending readmission. The clone buys chaos and self-bricking, not value creation. Confidentiality of the copied state is lost; integrity of the network is not.

Key Idea

Copying the software gives you the online identity and nothing in the offline domain. This is exactly the residue the hardware was hired to cover, and it is the whole of what it covers.

18 Rogue Firmware and the Measurement Gate

The next attack is to keep the real appliance and change its code. Reflash the RP2350 with firmware that ignores the one-prepared-record rule, signs stale positions, or exports witnesses before commit.

The host witness key was generated at birth under a seal whose unsealing predicate includes the enrolled firmware measurement. Firmware whose measurement differs from μ_{enrolled} receives an unsealing failure and cannot invoke the host signer. There is no fallback signing path, no debug bypass, and no host-provided override. Rogue firmware can still drive the chip, since the chip witness alone is not measurement-gated, and it can still produce the seed signature. It cannot produce the third, and the receiver needs all three.

This is why the measurement gate is described as load-bearing rather than as an implementation nicety. Firmware replacement does not give an attacker a programmable signing oracle; it gives them a brick.

Bitcoin Comparison

A Bitcoin hardware wallet's firmware is what stands between the key and the world. Replace it and the key signs whatever the new firmware asks.

Here the host key is sealed to the firmware measurement. Replace the firmware and the key does not unseal. What the attacker gains is a device that can no longer sign as itself.

Architectural consequence:

The single-appliance emission claim does not depend on the attacker being nice. It depends on the attacker being unable to run code the seal does not recognise, or unable to extract the key from the partition.

Tradeoff / boundary:

The RP2350 has publicly documented non-invasive fault-injection results, and the specification treats a measurement-boundary breach as outside the enrolled-appliance model, grouped with perfect cloning. This is an assumption about silicon, stated as one, and the open bench item V4 (Section 24) is its

validation.

19 The Restored Backup

An attacker, or an unlucky user, restores the appliance's host image from a backup taken several offline transfers ago. The host now believes it is at position u_{old} . The chip's counter has already consumed $H_0 - H > u_{\text{old}}$ steps and cannot be wound back.

Recovery's Ready branch sees $\text{Active}.u < H_0 - H$ and fails closed. Offline operation does not resume. Any counterparty holding a later frontier rejects the image's proposals on the frontier check regardless. Any authenticated diagnostic read exposes the desynchronisation.

The reverse direction, $\text{Active}.u > H_0 - H$, is impossible for an honest appliance unless the host is ahead of the chip, the wrong chip is attached, or a counter invariant has been violated; it does not resume offline operation either.

Key Idea

The physical counter cannot rewind, and the committed position cannot advance without it. A stale image is caught by comparing two integers before anything is signed.

20 The Perfect Clone and What Happens Later

Take the boundary case seriously. An attacker has extracted and perfectly emulated the exact current non-exportable state of all three factors, or has broken the measurement boundary so that multiple devices can produce accepted releases as if they were one enrolled appliance. Two releases from one origin now exist, both with three valid signatures, both accepted by strangers.

The specification does not claim strict offline uniqueness in this case. It claims what the software theorem still gives, which is more than any prior offline design gives.

They collide. Both releases commit position u_i+1 at the same leaf key under the same bundle B , with different successor roots. That is two claims to one position under one identity, decided by direct leaf comparison. The conflict is a decidable collision, not a suspicion.

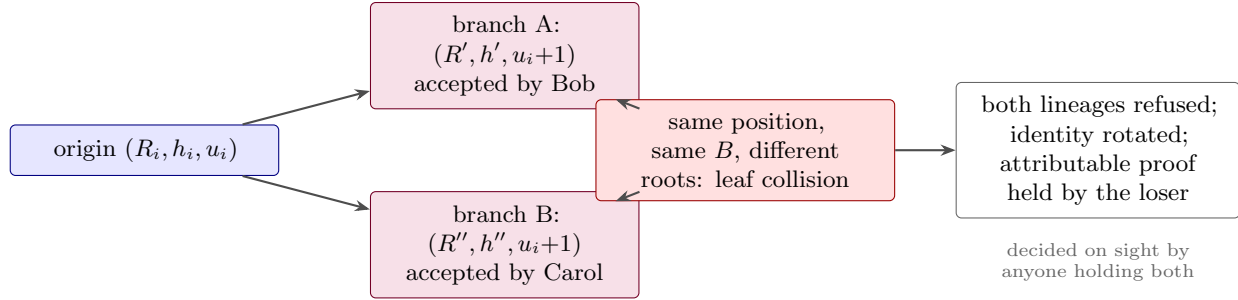
They cannot merge. Any receiver pinned to one branch rejects the other by frontier chaining, without knowing a split exists. Two incompatible successors of one consumed origin cannot both be represented as one coherent DSM history.

They propagate. Both colliding certificates replicate through every Branch Proof that crosses the divergence, so the set of parties who can decide the split grows with every transfer on either branch. Tripwire is the contact channel by which parties holding one branch or neither come to hold both.

They are attributable. The loser holds two witnessed state advances from one committed origin, each signed by the same chip-rooted identity, naming the same position with different roots. That is a proof of equivocation attributable to a physical device identity, verifiable by anyone who holds both objects.

They are bounded. If the physical counter discipline remains enforced, each committed release consumes one step and policy bounds steps between reconciliations by W . If the attacker also bypasses the counter, the system is outside the hardware identity model and must rotate authority.

They are bricked. Once the split is decided, honest counterparties refuse both lineages pending readmission under rotated authority. At most one branch is counter-consistent with the live chip, so a diagnostic read exposes at least one as desynchronised; it does not certify the survivor as honest, since the holder chooses which branch consumes decrements.



Bitcoin Comparison

When a Bitcoin double spend succeeds against an offline merchant, the outcome is final and anonymous: one transaction confirmed, the other vanished, and the merchant’s evidence is a signed transaction that anyone could have relayed.

When a DSM offline double spend is attempted through a perfect clone, neither branch can be adopted by anyone holding the other, the collision is decided by comparing two leaves, the evidence names a physical device, and the number of such releases between reconciliations was capped in advance.

Architectural consequence:

The clone boundary degrades from “strict single-appliance uniqueness” to “non-mergeable, exposed, bounded, and bricked.” It does not degrade to “silent loss.”

Tradeoff / boundary:

Tripwire exposes incompatible proposals at contact or reconciliation, not instantly across disconnected receivers. Between the two acceptances and the first comparison, both recipients believe they were paid. That interval is real, and its length is a property of connectivity, not of the protocol.

21 Compromise Matrix

Attacker holds	Online domain	Offline domain
phone / seed only	takeover	safe; lacks chip and host
chip only	safe	safe; lacks seed and host
host partition only	safe	safe; lacks seed and chip
phone + chip	takeover	safe; lacks host
phone + host	takeover	safe; lacks chip
chip + host	safe	safe; lacks seed
phone + chip + host, enrolled firmware intact	takeover	one live appliance; strict single- appliance uniqueness, no fork
phone + chip + host + measurement breach or perfect emulation	takeover	outside offline distinguishability; non-mergeable, exposed, bounded, bricked

Two rows deserve a sentence each. The seventh row is theft of the complete appliance together with the phone and seed. Under enrolled firmware, this gives the thief the ability to *operate* the single appliance, not to fork it; local unlock policy outside the specification is the mitigation. The eighth row is the boundary, and Section 20 is what the protocol still guarantees there.

Laser-assisted fault injection has been demonstrated against PUF constructions in laboratory settings. The design does not rest the offline identity on the PUF alone for exactly this reason: extracting the chip factor leaves the attacker without seed and host. Policy should treat confirmed physical compromise of the chip as grounds for authority rotation.

Part IV

Assessment

How this sits against everything that works, what is deliberately not introduced, what the silicon has shown so far, and the whole design in one picture.

22 Relation to Everything Else That Works

Each of these works, each has a boundary, and the design is best understood by where its boundary sits.

Chaum’s blind signatures, Chaum–Fiat–Naor, Brands [7, 8, 10]. A coin may be spent offline, and offline double spending is not prevented but *detected*: spend it once and stay anonymous, spend it twice and the deposit step reveals you. The boundary is the deposit, and everything between the second spend and the deposit is loss. DSM is not anonymity-first and does not defer to a deposit: uniqueness is enforced at acceptance as a property of the consumed state, and reconciliation exposure is the boundary case, not the mechanism.

The Chaum–Pedersen observer, smartcard purses, offline CBDC [9, 12, 13]. A tamper-resistant chip co-signs each payment and refuses the same value twice. The chip is the transaction authority; break it and the money is broken. DSM inverts this: the chip proves identity, uniqueness is a theorem that reads no chip, and under the enrolled firmware one appliance emits one release per origin because of *when* the witness is minted, not because of a rule the chip is trusted to keep.

Bitcoin hardware wallets. Protect a key. Do not, and do not claim to, refuse the second signature. DSM’s appliance does refuse it, by timing against a one-way counter, and the key it protects is one of three.

Lightning [11]. Moves the common case off-chain but settles disputes by broadcasting a signed state to the chain within a timeout, so safety reduces to the ledger and to being awake. DSM has no ledger to reduce to, no timeout, and no watchtower; a stalled counterparty is an availability failure, not a safety one.

	Uniqueness au- thority	Chip’s job	Two strangers, one coin	Break the chip and...
eCash / Brands	issuer at deposit	none	both accept; caught at deposit	n/a
Observer / purse / CBDC	the chip	enforce no-double-spend	refused by the chip	double spend
Lightning	the chain	protect a key	needs the chain	key theft
Hardware wallet	the chain	protect a key	chain decides	key theft
DSM offline bearer	the state predicate	prove identity; one release per origin by timing	refused by the appliance; if cloned, collides and bricks	forge a device, not a transfer

23 What Is Not Introduced, and What Is Not Claimed

Deliberately not introduced

No chip as transaction authority. No live counter read in acceptance. No relay session from receiver to payer’s chip. No counter-disclosure round trip before a first transfer. No MAC-and-destroy witness; its slot state is a pure function of its input and is host-restorable, so it carries no lineage. No fused anchor head and no boot-ticket chain as protocol objects. No receiver-side use of the firmware measurement; it survives only as an unsealing gate inside the appliance. No fork-gossip layer. No two-of-three. No hardware requirement for online DSM. And no rule that a hardware counter decides whether a transfer is unique.

Deliberately not claimed

That a perfect live emulation of all three factors can be distinguished offline. That a measurement-boundary breach on the RP2350 leaves the single-appliance emission claim intact. That the PUF cannot be attacked by fault injection. That the online domain survives phone compromise. That Tripwire notifies disconnected receivers instantly. That the counter cap bounds value rather than count. That chip signatures are stable byte strings. That the July 2026 silicon runs constitute end-to-end validation of three-signature acceptance, the counter initialisation lock, or the measurement-gated host signing path; those are the open bench items. Or that conformance follows merely because the primitives exist in a repository.

24 What the Silicon Has Shown

The design targets a Raspberry Pi Pico 2 W (RP2350) as host, a TROPIC01 secure element on a MikroElektronika Click board over SPI as the anchor, and nothing else. Two bench runs on that hardware are retained and reinterpreted under the current claims.

July 8, 2026: counter discipline. On two physical appliances, one with a used chip and one production-fresh, the run validated counter birth (a virgin counter initialised to maximum and adopted as H_0), that Prepare leaves the counter untouched, that Commit performs exactly one decrement, that a second commit against the same prepared transition is refused with the counter stable, that $u = H_0 - H$ held at every observation, and that the two dies presented distinct static public identities. What the run was originally built to demonstrate, receiver-witnessed before-and-after counter reads, is no longer part of the protocol; those reads survive only as diagnostic audit.

July 9, 2026: resident Ed25519 gate. On a fresh board with an untouched slot, an empty-slot read failed cleanly, in-slot Ed25519 key generation succeeded, on-die signatures verified off-chip against the exported public key, erase was reversible, the public key persisted byte-identically across power cycle, and signatures over the same key and message differed while both verified. The counter was not touched. This retires the resident-witness-key bench item.

Open. End-to-end three-signature release acceptance against the reference verifier (V2); counter initialisation lock behaviour where the hardware exposes it (V3); and the measurement-gated host signing boundary, including that rogue firmware receives an unsealing failure and that no export is possible before commit (V4).

Bitcoin Comparison

Bitcoin’s safety argument is empirical and economic: the chain has held under adversarial conditions for a long time, and the cost of rewriting it is public.

DSM’s offline safety argument is a software theorem for uniqueness, an appliance-emission theorem with its assumptions written down, and a bench record that is honest about what has and has not been exercised.

Architectural consequence:

A reader can see exactly which claim rests on which assumption, and which assumptions have been touched by silicon.

Tradeoff / boundary:

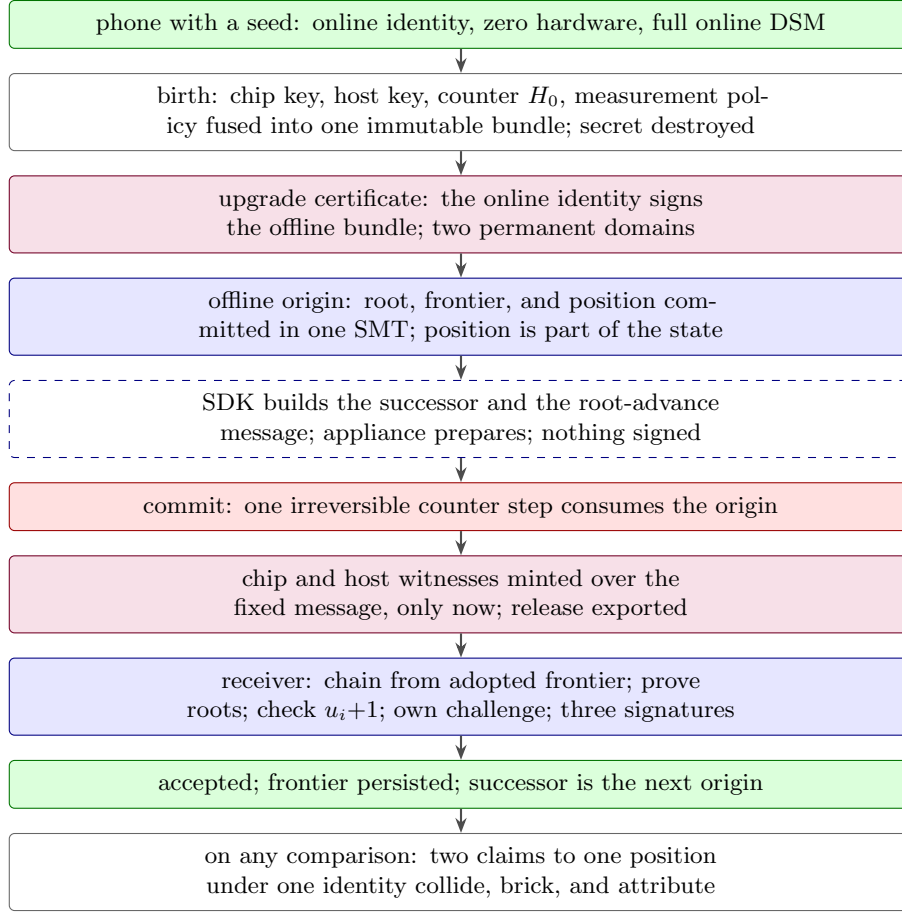
A proof about a model says nothing about a bug in the firmware that implements it, and two bench runs are not a decade of adversarial uptime. The honest position is to want both.

25 Security Properties

Property	Statement
Adopted-frontier exclusion	No honest receiver accepts and executes two successors of one adopted origin; rejection is up front and reads no hardware
Single-appliance emission	Under enrolled firmware, one appliance cannot emit two distinct valid releases from one origin, even to first-contact strangers
Clone exclusion	A party lacking any one of the three factors produces no accepted offline release
Online-domain scope	Phone compromise yields the online domain only
Birth non-recreation	Public enrollment data cannot recreate the offline identity on new hardware
Stale-image rejection	A restored earlier image cannot resume offline operation
Split bricking	Two lineages of one identity collide at one position; each receiver rejects the other branch; both are refused once the split is decided
Replay idempotence	Re-emitting the same package is duplicate delivery, not a second spend
Recoverable commit	Recovery re-emits the same release or downgrades; it never signs a different release for the same counter step
Residual bound	Past the measurement boundary, incompatible histories remain non-mergeable, exposed, bounded by W while counter discipline holds, and bricked

The first is a theorem of the DSM kernel and needs none of the hardware. The second rests on non-exported keys, a measurement-gated seal, witnesses minted after the counter step, and single-threaded enrolled firmware. Everything below rests on the same four things plus collision resistance of H and unforgeability of the three signature schemes. The TLA+ sketch in the specification checks the software uniqueness invariant, the counter synchronisation invariant, and the single-appliance emission invariant over an honest single-threaded appliance; it assumes the primitives rather than modelling a forgery adversary.

26 The Complete Causal Architecture



27 Final Invariant

The complete offline safety relation, compactly. An offline release from origin (R_i, h_i, u_i) is accepted by receiver ρ only if

$$\begin{aligned}
 \text{Accept}_{\text{off}} \implies & \text{ChainsFrom}(\text{frontier}_{\rho}) \wedge R_i \ni (B, h_i, u_i) \wedge R_{i+1} \ni (B, h_{i+1}, u_i+1) \\
 & \wedge h_{i+1} = H(h_i \parallel D_{i+1}) \wedge r_R \text{ chosen by } \rho \wedge \text{recipient} = \rho \\
 & \wedge \text{DsmVerify}(\Delta_{i+1}^{\circ}) \wedge \text{ChipVerify}(pk_{\text{chip}}, M_{i+1}, \sigma^{\text{chip}}) \wedge \text{HostVerify}(pk_{\text{host}}, M_{i+1}, \sigma^{\text{host}}),
 \end{aligned}$$

and any σ^{host} that exists for the origin was minted after the one counter step that consumed it. Consequently, for one adopted origin at one honest receiver, at most one successor is executed; for one enrolled appliance under its measurement gate, at most one release per origin is emitted; and past that gate, two releases collide at one position and cannot both be history.

*The chip never decides whether a transfer is unique.
 The software never tries to prove the device is real.
 No witness exists while the origin is still spendable.*

state $\rightarrow$ prepare $\rightarrow$ consume $\rightarrow$ witness $\rightarrow$ emit $\rightarrow$ verify $\rightarrow$ accept

28 Conclusion

The previous offline specification asked hardware to do what software already did, and the machinery required to make the hardware appear to do it, receiver-witnessed counter reads, MAC-and-destroy witnesses, fused heads, boot tickets, disclosure round trips, was the bulk of the document. Removing the misassignment removed the machinery.

What remains is small. DSM consumes state whole; one parent, one executed successor at an honest acceptance point; that is transfer uniqueness, and it is software. Identity is two domains: a seed for the online world, at zero hardware cost, and a three-factor fusion of seed, PUF-rooted chip key, and measurement-gated host key for the offline world, in which every release is witnessed by all three. The SMT counter makes offline position explicit inside the consumed state. The physical counter is a floor, a tripwire input, and a budget. Tripwire exposes incompatible proposals and bricks whatever splits, but the safety property is already enforced at acceptance. The measurement gate adds the appliance boundary: enrolled firmware emits one committed successor per frontier, rogue firmware cannot unseal the host signer, and witnesses are minted only after the one-way counter step, so no signed release for an origin exists while that origin is still spendable.

Every offline-cash design before this one made the chip the judge and hoped the chip would hold. This one lets the chip say who it is, and lets the state decide the rest. That is not a hardware authority with a software veneer. It is a software authority with hardware identity, which is what a Deterministic State Machine required all along.

References

- [1] B. Ramsay. *Software Authority, Hardware Identity*. Irrefutable Labs Inc., July 2026.
- [2] B. Ramsay. *Deterministic State Machine: A Concise, Post-Quantum Specification*. Irrefutable Labs Inc.
- [3] B. Ramsay. *Deterministic State Machines as Guarded Linear Constraint Systems*. Irrefutable Labs Inc., 2026.
- [4] B. Ramsay. *Deterministic State Machines Explained Mathematically: Bilateral State, Sparse Merkle Trees, Determinism, Precommitment, Linear Resources, Canonical State Chaining, and Conflict-Local Finality*. Irrefutable Labs Inc., 2026.
- [5] B. Ramsay. *The dBTC Is the Asset; the Bitcoin Key Is Machinery — dBTC: A High-Level Explainer for Bitcoin Readers*. Irrefutable Labs Inc., September 2026.
- [6] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
- [7] D. Chaum. Blind signatures for untraceable payments. *CRYPTO 1982*.
- [8] D. Chaum, A. Fiat, M. Naor. Untraceable electronic cash. *CRYPTO 1988*.
- [9] D. Chaum, T. P. Pedersen. Wallet databases with observers. *CRYPTO 1992*.
- [10] S. Brands. Untraceable off-line cash in wallet with observers. *CRYPTO 1993*.
- [11] J. Poon, T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. 2016.
- [12] R. Auer, R. Böhme. *The Technology of Retail Central Bank Digital Currency*. BIS Working Paper, 2020.
- [13] L. Kempen, J. Pouwelse. *Offline Digital Euro: a Minimum Viable CBDC using Groth-Sahai proofs*. arXiv:2407.13776, 2024.
- [14] Tropic Square. *TROPIC01 Datasheet; TROPIC01 Physical Unclonable Function (PUF)*, ODR TR01 PSR 001, v1.0, June 2026.
- [15] Raspberry Pi Ltd. *RP2350 Datasheet*.